

Projektdokumentation - F1 Team-Manager

Modul 295 Armend Morina / Donat Recica

1. Projektidee

Der F1 Team-Manager ist ein Backend (REST-API), mit dem man Formel-1-Teams und ihre Fahrer verwalten kann. Ein Team hat einen Namen und einen Standort und dazu mehrere Fahrer. Jeder Fahrer gehört zu genau einem Team (eine Eins-zu-viele-Beziehung).

Gedacht ist die Anwendung für jemanden, der den Überblick über seine Teams und Fahrer behalten will, zum Beispiel als Grundlage für eine spätere Weboberfläche. Man kann Teams anlegen, ansehen, ändern und löschen (das volle CRUD). Die Daten werden dauerhaft in einer PostgreSQL-Datenbank gespeichert, damit sie auch nach einem Neustart erhalten bleiben.

2. Anforderungen (User Stories)

User Story 1 - Team anlegen Als Team-Manager möchte ich ein neues Team mit seinen Fahrern anlegen, damit ich meinen Rennstall im System habe. Akzeptanzkriterien: - Ich schicke Name, Standort und die Fahrer. - Das System speichert das Team und vergibt automatisch eine id. - Leere Pflichtfelder werden abgelehnt (klare Fehlermeldung, kein Absturz).

User Story 2 - Teams ansehen Als Team-Manager möchte ich alle Teams mit ihren Fahrern sehen, damit ich einen Überblick habe. Akzeptanzkriterien: - Ich bekomme eine Liste aller Teams. - Zu jedem Team sehe ich die zugehörigen Fahrer. - Ich kann auch ein einzelnes Team über seine id abrufen.

User Story 3 - Team löschen Als Team-Manager möchte ich ein Team löschen, wenn ich es nicht mehr brauche. Akzeptanzkriterien: - Ich lösche über die id. - Das Team und seine Fahrer verschwinden aus der Datenbank. - Bei einer id, die es nicht gibt, kommt eine saubere Fehlermeldung (404), kein Absturz.

3. Klassendiagramm

Das Modell besteht aus zwei Klassen mit einer Eins-zu-viele-Beziehung:

- **Team** - Attribute: id, name, base, drivers (die Liste der Fahrer)
- **Driver** - Attribute: id, firstName, lastName, number, points, team

Ein Team hat viele Fahrer (@OneToMany), ein Fahrer gehört zu einem Team (@ManyToOne). Der Fremdschlüssel team_id liegt in der Tabelle drivers.

Hinweis: Das saubere Diagramm als Bild aus draw.io hier einsetzen.

4. Projektstruktur (Ordner und Dateien)

Das Projekt ist nach der 3-Schichten-Architektur aufgebaut (Controller, Service, Repository). Jede Schicht hat ihr eigenes Package.

```
f1teammanager/  
├── pom.xml                Maven-Konfiguration (Abhängigkeiten, Java-Version)  
├── docker-compose.yml     startet die PostgreSQL-Datenbank  
├── README.md              Kurzbeschreibung und Startanleitung  
└── src/  
    ├── main/java/ch/wiss/f1teammanager/  
    │   ├── F1teammanagerApplication.java  Startpunkt der Anwendung  
    │   ├── DataSeeder.java                legt beim Start Beispiel-Teams an  
    │   ├── model/                         Team.java, Driver.java  
    │   └── dto/                           TeamDTO, DriverDTO, TeamFormDTO, DriverFormDTO, ErrorR  
    ├── response  
    │   ├── mapper/                       TeamMapper.java  
    │   ├── repository/                   TeamRepository.java  
    │   ├── service/                      TeamService.java  
    │   ├── controller/                   TeamController.java  
    │   └── exception/                    TeamNotFoundException.java, GlobalExceptionHandler.jav  
    └── test  
        ├── main/resources/application.properties  Datenbank-Verbindung  
        └── test/java/ch/wiss/f1teammanager/      die Unit-Tests
```

Alle Klassen und ihre Aufgabe:

Klasse	Package	Aufgabe
F1teammanagerApplication	(Basis)	Startpunkt, fährt die Spring-

Klasse	Package	Aufgabe
		Boot-App hoch
DataSeeder	(Basis)	füllt die Datenbank beim Start mit Beispiel-Teams
Team	model	Entity Team, hat die Fahrer-Liste (@OneToMany)
Driver	model	Entity Fahrer, gehört zu einem Team (@ManyToOne)
TeamRepository	repository	Datenbankzugriff (fertige Methoden von Spring Data)
TeamDTO	dto	Ausgabe eines Teams mit seinen Fahrern
DriverDTO	dto	Ausgabe eines Fahrers (ohne team-Feld)
TeamFormDTO	dto	Eingabe beim Anlegen und Ändern eines Teams
DriverFormDTO	dto	Eingabe eines Fahrers
ErrorResponse	dto	einheitliches Fehler-Format
TeamMapper	mapper	wandelt zwischen Entity und DTO um
TeamService	service	die Geschäftslogik, gibt DTOs zurück
TeamController	controller	die /api/teams-Endpoints (CRUD)
TeamNotFoundException	exception	eigener Fehler, wenn ein Team nicht existiert
GlobalExceptionHandler	exception	fängt Fehler zentral ab und macht saubere Antworten
TeamRepositoryTest	test/repository	Repository-Test (@DataJpaTest)
TeamServiceTest	test/service	Service-Test (Mockito)
TeamControllerTest	test/controller	Controller-Test (@WebMvcTest)

5. Ablauf je User Story (Folge von API-Aufrufen)

US1 - Team anlegen: POST /api/teams (mit den Team-Daten) → als Kontrolle GET /api/teams/{id} (zeigt das neu angelegte Team).

US2 - Teams ansehen: GET /api/teams (Liste aller Teams) bzw. GET /api/teams/{id} (ein einzelnes Team).

US3 - Team löschen: DELETE /api/teams/{id} → danach GET /api/teams (das Team ist nicht mehr in der Liste).

6. Datentypen je Endpoint (Beispiel-JSON)

POST /api/teams - Request:

```
{
  "name": "Red Bull Racing",
  "base": "Milton Keynes",
  "drivers": [
    { "firstName": "Max", "lastName": "Verstappen", "number": 1, "points": 0 }
  ]
}
```

Response (201):

```
{
  "id": 1,
  "name": "Red Bull Racing",
  "base": "Milton Keynes",
  "drivers": [
    { "id": 1, "firstName": "Max", "lastName": "Verstappen", "number": 1, "points": 0 }
  ]
}
```

GET /api/teams - Response (200): eine Liste solcher Team-Objekte.

Fehlerbeispiel (400, leerer Name):

```
{ "timestamp": "...", "status": 400, "message": "Validierung fehlgeschlagen",
  "fieldErrors": { "name": "darf nicht leer sein" } }
```

Fehlerbeispiel (404, id gibt es nicht):

```
{ "timestamp": "...", "status": 404, "message": "Kein Team mit der id 999 gefunden!" }
```

7. Testplan (5 manuelle Testfälle mit Insomnia)

#	Anfrage	Eingabe	Erwartetes Ergebnis (Soll)
1	GET /api/teams	-	200, Liste aller Teams mit Fahrern
2	POST /api/teams	gültiges Team (Name, Base, Fahrer)	201, neues Team mit id
3	POST /api/teams	leerer Name	400, Meldung "darf nicht leer sein"
4	GET /api/teams/999	-	404, "Kein Team mit der id 999 gefunden!"
5	DELETE /api/teams/{id}	gültige id	204, Team gelöscht

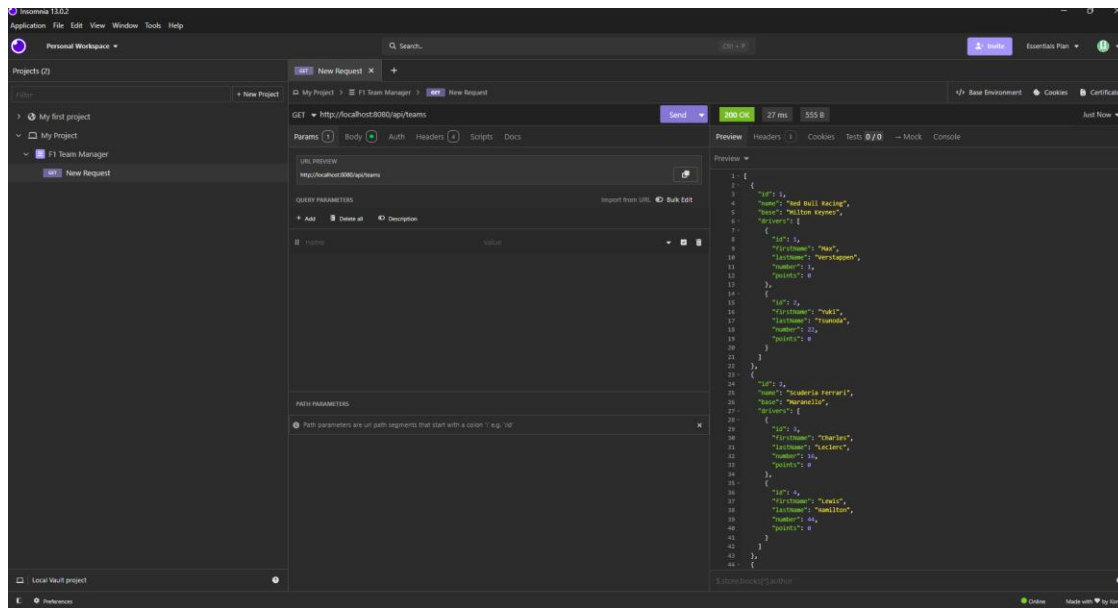
8. Testdurchführung (Soll/Ist)

Alle fünf Testfälle wurden mit Insomnia ausgeführt. Das Ist-Ergebnis entspricht in jedem Fall dem Soll.

#	Soll	Ist	ok?
1	200, Liste aller Teams	200, Liste mit Red Bull und Ferrari inkl. Fahrern	ja
2	201, neues Team mit id	201, McLaren angelegt, kommt mit id zurück	ja
3	400, Validierungsfehler	400, "Validierung fehlgeschlagen", Feld name "darf nicht leer sein"	ja
4	404, nicht gefunden	404, "Kein Team mit der id 999 gefunden!"	ja
5	204, gelöscht	204 No Content, Team entfernt	ja

Testfall 1 - GET /api/teams

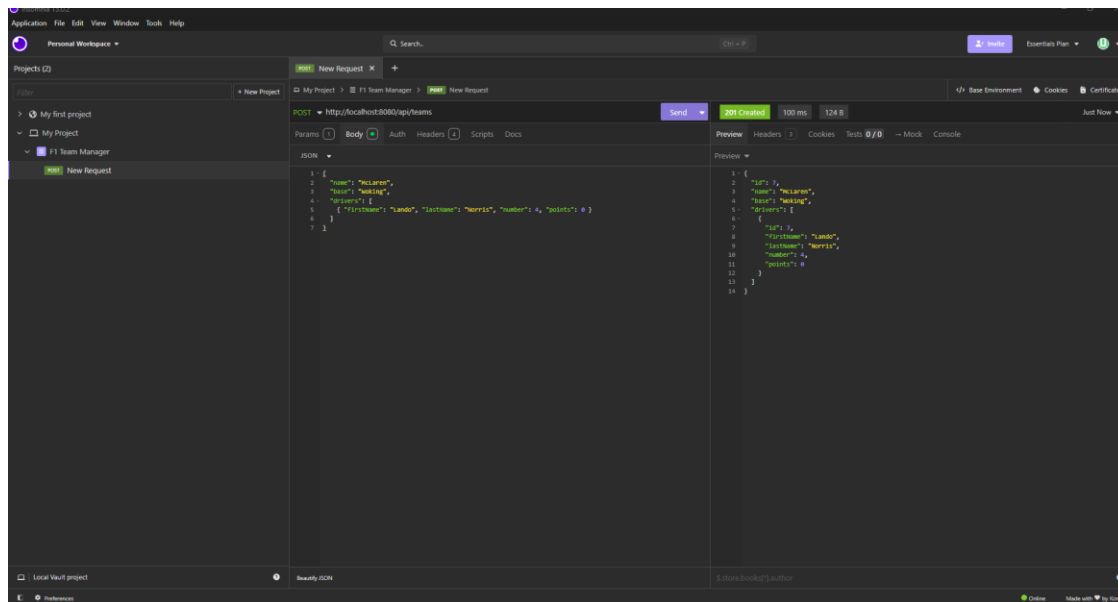
Ruft alle Teams ab. Antwort **200 OK** mit der Liste aller Teams und ihrer Fahrer.



Testfall 1 - GET alle Teams, 200 OK

Testfall 2 - POST /api/teams (gültiges Team)

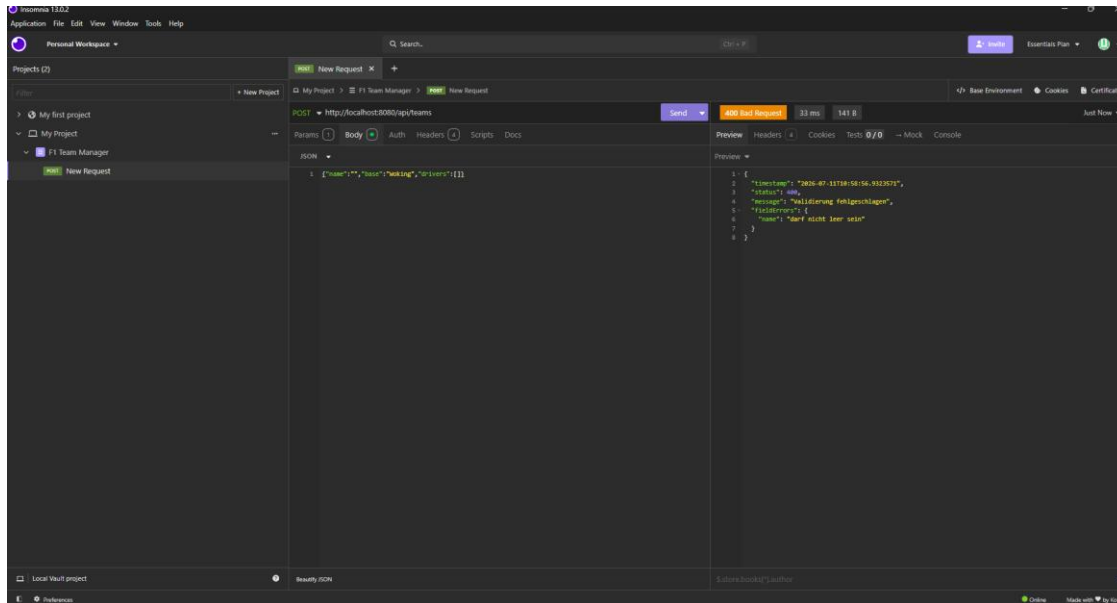
Legt das Team McLaren mit dem Fahrer Lando Norris an. Antwort **201 Created**, das Team kommt mit vergebener id zurück.



Testfall 2 - POST gültiges Team, 201 Created

Testfall 3 - POST /api/teams (leerer Name)

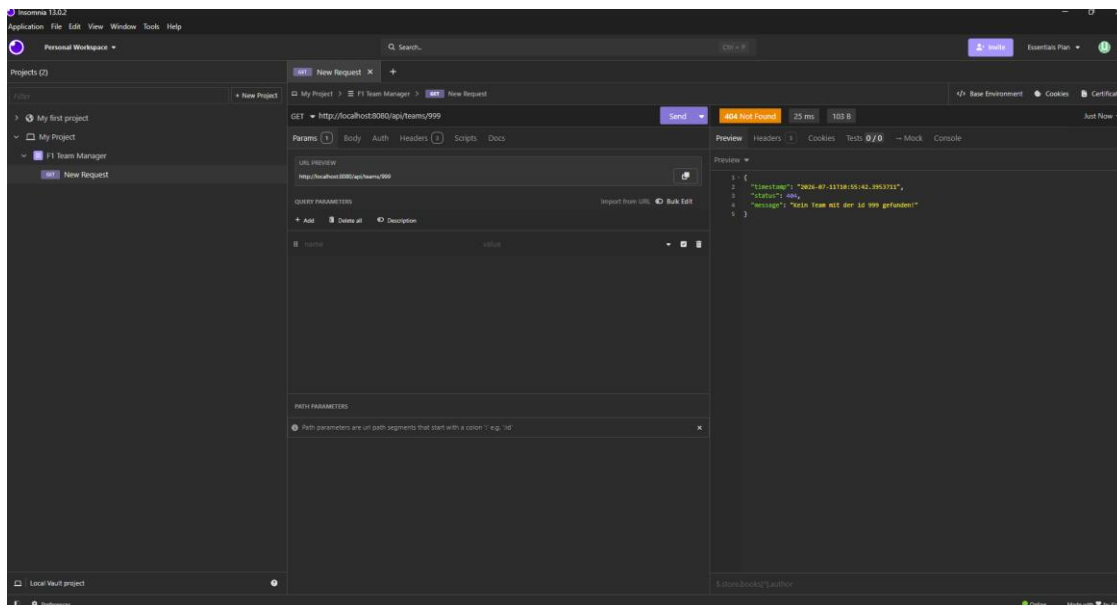
Schickt ein Team mit leerem Namen. Die Validierung greift: Antwort **400 Bad Request** mit der Meldung “darf nicht leer sein” beim Feld name.



Testfall 3 - POST leerer Name, 400 Bad Request

Testfall 4 - GET /api/teams/999

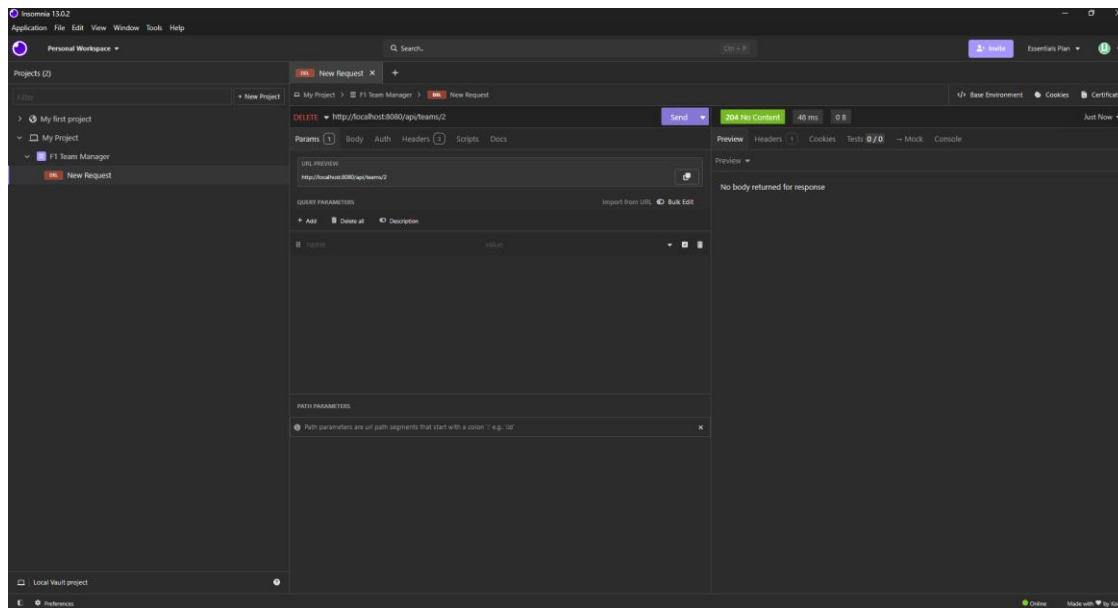
Fragt ein Team ab, das es nicht gibt. Antwort **404 Not Found** mit der Meldung “Kein Team mit der id 999 gefunden!”.



Testfall 4 - GET nicht existierende id, 404

Testfall 5 - DELETE /api/teams/2

Löscht das Team mit der id 2. Antwort **204 No Content** (erfolgreich, ohne Rückgabe).



Testfall 5 - DELETE Team, 204 No Content

9. Installationsanleitung

Voraussetzungen: Docker Desktop, ein JDK (21 oder neuer), IntelliJ IDEA (oder nur die Kommandozeile).

1. Das Projekt öffnen (Ordner f1teammanager in IntelliJ; Maven lädt die Abhängigkeiten automatisch).
2. Docker Desktop starten und warten, bis es läuft.
3. Im Projektordner die Datenbank hochfahren: `docker compose up -d`. Das startet den PostgreSQL-Container (Datenbank f1db, Benutzer f1User, Passwort f1PW, Port 5432).
4. Die App starten: in IntelliJ auf Run, oder auf der Kommandozeile `mvnw.cmd spring-boot:run`.
5. Prüfen: `http://localhost:8080/api/teams` aufrufen. Es erscheinen zwei Beispiel-Teams (der DataSeeder füllt die leere Datenbank beim ersten Start automatisch).
6. Optional die Tests laufen lassen: `mvnw.cmd test` (die Datenbank muss dafür laufen).

10. Hilfestellungen und Quellen

- **Kursunterlagen Modul 295** (Blöcke 1 bis 7, inkl. dem Referenzprojekt Quiz-Backend) als Vorlage für Aufbau und Muster.

- **KI-Hilfe (Claude):** Der Code ist grösstenteils mit Hilfe einer KI entstanden. Damit ich jede Zeile selbst verstehe und erklären kann, habe ich zu jeder Klasse, Funktion und jedem Begriff eine eigene Lern-Doku geführt (DOKUMENTATION.md), in der alles Schritt für Schritt erklärt ist. Verwendet wurden nur die im Kurs behandelten Techniken.